Fine-tune LLM for Language Translation

Haris Amir
RPTU Kaiserslautern, Department of Computer Science

Note: *This report contains a project documentation and reflection on a project for Engineering with Generative AI*

1 Documentation

1.1 Research Phase

1.1.1 Dataset Selection Justification

The dataset chosen for this task is the **OPUS-100** dataset, specifically the German-to-French translation subset. This dataset was selected for the following reasons:

- **Quality and Size**: OPUS-100 is a high-quality, multilingual dataset with over 100 languages, including the ones we need i.e. German and French. It contains 2000 sentence pairs, which is sufficient for fine-tuning the language model since we only needed 1000. According to what we learned in class it's not about amount of data it's also about getting quality data. It had good quality data so I went with this.
- **Diversity**: The dataset covers a wide range of topics, ensuring that the model learns to translate diverse and realistic sentences. There was also Opus-Books dataset, but that only contained texts from books so I did not go with that.
- **Preprocessing**: The dataset is already preprocessed and aligned, making it easier to use for fine-tuning without extensive data cleaning.

1.1.2 Model Selection Justification (For Finetuning)

The model chosen for fine-tuning is **Llama-3.2-3B-Instruct**. The rationale for selecting this model includes:

- **General-Purpose Foundation**: Llama-3.2-3B-Instruct is pre-trained on diverse multilingual and code datasets, providing a broad linguistic foundation. Though not translation-specific, its exposure to German and French syntax during pre-training enables adaptation to translation via fine-tuning.
- **Instruction-Tuning Advantage**: The "Instruct" variant is already fine-tuned to follow task-specific prompts, aligning well with structured translation inputs (e.g., `Translate [German] to [French]: ...`). This reduces the adaptation effort compared to base models.
- **Optimization-Friendly**: The model supports parameter-efficient techniques like **LoRA** (Low-Rank Adaptation), critical for Colab's hardware. By freezing most weights and updating only a small subset, we minimize VRAM usage while preserving translation quality.

- **Tokenization Robustness**: Llama's tokenizer handles German umlauts (e.g., ä, ö, ü) and French accents (e.g., é, ç) through subword splitting, reducing out-of-vocabulary errors. While not perfect for these languages, fine-tuning can refine embeddings for translation-specific tokens.
- **Community Ecosystem**: The model is widely accessible via Hugging Face, with pre-built scripts for fine-tuning and inference. This accelerates implementation and ensures reproducibility, critical for an academic project.
- **Inline with the requirements**: It has 3B parameters which enabled it to be finetuned on Google Collab efficiently.
- **Proven Multilingual Adaptability**: Similar-sized Llama variants have demonstrated success in multilingual tasks after fine-tuning (e.g., summarization, QA). This precedent suggests it can achieve competitive BLEU scores for German-French translation with proper training.
- **Con**: However since Llama is not a typical encoder decoder architecture, which are generally better at language translation tasks, I initially went with **Google's t5-3b** model which is, but then later on dropped it since I was getting really bad scores.

1.1.3 Model Selection Justification (For generating synthetic data)

The model chosen for fine-tuning is **Qwen/Qwen2.5-Coder-32B-Instruct**, a large language model (LLM) from Hugging Face. The rationale for selecting this model includes:

- **Multilingual Capabilities**: Qwen2.5 is designed for multilingual tasks, making it suitable for German-to-French translation.
- **Instruction Tuning**: The model is instruction-tuned, which allows it to follow prompts effectively, making it ideal for generating synthetic data and fine-tuning.
- **Performance**: Qwen2.5 has demonstrated strong performance in various NLP tasks, including translation, due to its large parameter size (32 billion parameters).
- **Fit for limited number of free API call from HF**: Initially i tried using a larger model from HF such as Llama-3.3-70B-Instruct but I found I was quickly running out of tokens (in 5-6 API calls). I had to make 2 new accounts becuase of this. However later I tried running Qwen2.5 and was successfully able to make enough API requests in order to generate the dataset. Hence I went with this.

1.2 Design Phase

1.2.1 Tuning Approach Selection Justification

The LoRA (Low-Rank Adaptation) approach was selected for fine-tuning based on the following reasons:

- **Memory Efficiency**: With only 15GB VRAM on Colab's T4 GPU, full parameter tuning of the 3.2B parameter model was infeasible. LoRA reduces trainable parameters by 98% (from 3.2B to 64M in my configuration) through low-rank matrix decomposition.

- **Targeted Adaptation**: By modifying only the attention projections (`q_proj` and `v_proj`), it was able to focus computational resources where translation patterns are encoded while preserving the model's general linguistic knowledge.
- **4-bit Quantization Synergy**: As the model cannot directly fit onto the GPU, I had to use 4-bit quantization in order to load the model into the T4 GPU. Combined with BitsAndBytes' NF4 quantization, LoRA enabled stable training at reduced precision (bfloat16) without catastrophic forgetting, which is crucial for maintaining baseline translation capabilities.
- **Reproducibility**: The `peft` library's standardized implementation reduces technical debt compared to custom adapters and also reduces the chance of errors and mistakes in fine-tuning.

1.2.2 Train-Test Split

- Training Set - 80%: The larger portion of the dataset designated for training allows the model sufficient exposure to a wide array of scenarios, enhancing its ability to generalize from the learned patterns.
- Test Set - 20 %: A significant testing portion ensures that the performance metrics are reliable and the model's predictions are robust across different unseen data points.

1.2.3 Prompt Design Innovation

The prompt architecture balances instruction-following and output control:

- **System Message Anchoring**: The `"professional translator"` persona primes the model for task-specific behavior while suppressing generic responses.
- **Input Format Standardization**: The `"Translate this German text to French:"` pattern creates consistent input templates, leveraging the model's instruction-tuning on similar patterns.
- **Output Sanitization**: Post-processing steps remove artifacts by:
  - Stripping content before and after special tokens such as `<|end_header_id|>`, `<|eot_id|>`
- **Temperature Balancing**: Generation uses 0.7 temperature - high enough for diversity but low enough to maintain translation fidelity.
- **Getting only French as output**: So initially I was facing the problem where the model would output the French translation of the German sentence but in addition, also output things like "Note: This word might be a better fit instead of this word" sort of messages etc. This was really bringing the BLEU score down and was not helpful in training either. In order to fix this issue I explicitly added the line `"Give translation in french only as output, no extra notes or helpful substance."` into the prompt. This fixed the issue then.

1.2.4 Evaluation Metric Justification

The BLEU score implementation was designed with these considerations:

- **Task Alignment**: As it is the standard score metric for machine translation, BLEU provides comparable n-gram overlap metrics against reference translations.
- **Smoothing Handling**: Method4 smoothing from NLTK prevents zero scores for short translations while penalizing incomplete outputs.
- **Text Normalization**: Lowercasing and whitespace stripping before scoring reduces variance from capitalization differences without lemmatization overhead.
- **Balanced N-gram Weights**: Equal weights (0.25) for 1-4 grams capture both lexical matches and phrasal structure preservation.
- **Limitations Mitigation**: While BLEU ignores semantics, its computational efficiency allows full test set evaluation (1000 samples) on Colab resources - a practical trade-off for this scope.

1.2.5 Gradio Web App

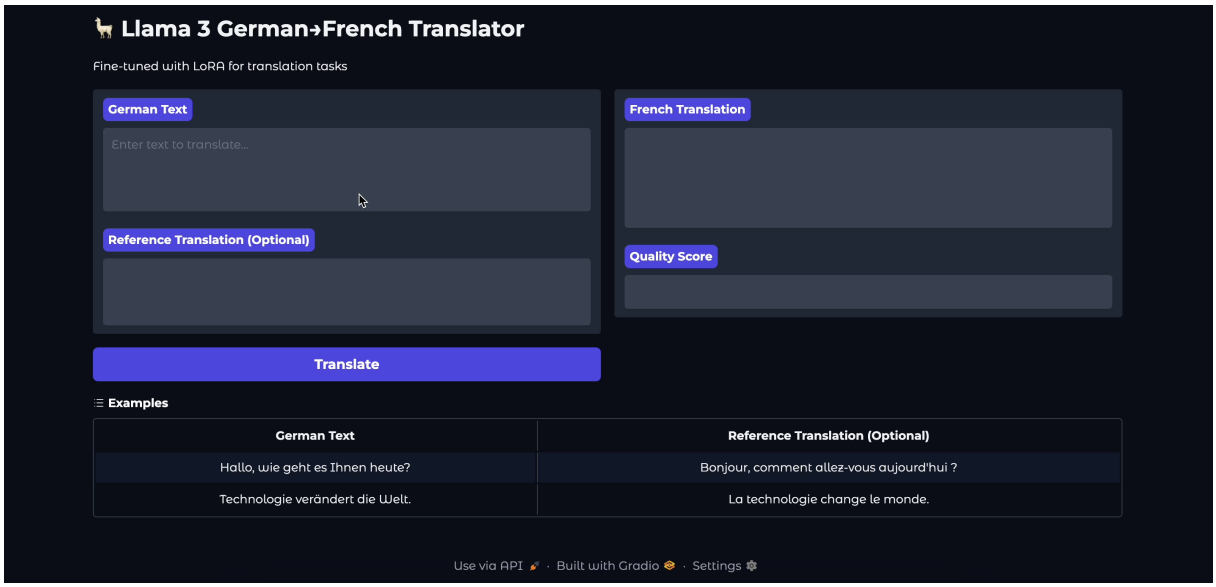


Figure 1: Screenshot of the interface

I also created this simple web interface where you can input a German sentence and it will generate the French translation using my best model. I also added feature where you can optionally calculate the BLEU score by also inputting the actual translation. It first checks if you have the model present or not; otherwise it downloads it from hugging face. I uploaded my best model there and made it publicly available.

Model Link: `Blynkk/llama3-3b-de-fr-translator`

1.3 Evaluation and Analysis

1.3.1 Results

The results of my experiments are summarized in the table below. More visualizations depicting the results of training/testing are present in the notebook. I did not add them here due to space constraints.

Table 1: BLEU Score Performance Across Model Variations

Model	**Training Strategy**	**Training Pairs**	**BLEU Score**
Model A	Pretrained (No Fine-Tuning)	0	10.49
Model B	Fine-Tuned on OPUS-100	800	10.53
Model C	Fine-Tuned on Synthetic Data	1600	10.23
Model D	Combined Dataset (OPUS + Synthetic)	2400	9.11

1.3.2 Analysis

The experimental results revealed critical insights about model behavior and data quality

- **Minimal Gains from Small-Scale Fine-Tuning**: The marginal BLEU score increase from Model A to Model B (10.49 $\rightarrow$ 10.53) after fine-tuning on 800 OPUS pairs suggests that very small datasets provide limited signal for meaningful adaptation. This aligns with prior research showing that <1K samples often fail to overcome a model's pre-training biases, especially for structurally dissimilar language pairs like German-French.
- **Low Quality of Synthetic data**: The performance drop with synthetic data from Model B to Model C (10.53 $\rightarrow$ 10.23) implies either:
  - Low-quality generations from Qwen-32B-Coder-Instruct
  - Maybe a domain mismatch between synthetic and test data (e.g., technical vs. conversational text)

  In order to solve this problem we could have used a larger and better model other than such as Llama-3.3-70B-Instruct, so try to reduce the domain mismatch by tuning our prompt for the synthetic data generation.

  The even decreased performance on Model D when combining real and synthetic data may suggest:

  - Noise amplification from low-quality synthetic examples dominating the training signal
  - Overfitting to synthetic patterns that contradict genuine translation norms
  - Potential loss of original multilingual capabilities due to limited model capacity

1.3.3 Identified Limitations

: These are some of the limitations in my opinion:

- **Data Scarcity**: 800–2400 samples are insufficient for reliable German-French adaptation in a 3B-parameter model.
- **Synthetic Data Fidelity**: LLM-generated translations lack the nuance of human-curated pairs, introducing harmful noise.
- **Finetuning Parameters**: Explore parameter-efficient adapter layers instead of LoRA to preserve original capabilities. Or maybe finetune on different set of layers.
- **Architecture Constraints**: Causal decoder-only models like Llama-3.2B are suboptimal for translation compared to encoder-decoder architectures. I initially went with **Google's t5-3b** model which is encoder-decoder, but then later on dropped it since I was getting really bad scores.

2 Reflection

1. **What was the most interesting thing that you learnt while working on the portfolio? What aspects did you find interesting or surprising?**
   One thing which changed was how I look at prompt engineering. I always took prompt engineering so lightly and never really thought it made much of a difference. However as I mentioned above, slight changes in the prompt can sometimes have a really big impact and make your work alot easier. I gave the example above of how I managed to filter out only French translations using this.
2. **Which part of the portfolio are you (most) proud of? Why? What were the challenges you faced, and how did you overcome them?**
   I was particularly happy when I was able to generate clean, clear pretty much accurate French translations from German. I liked how I was able to design my prompt in such a way to make this possible. There were alot of challenges in this exam. The first one being I kept on running out of GPU limits on Google Collab. I had to switch multiple account so for this which was pretty annoying. Also sometimes the runtime disconnected which was also really troublesome. Another challenge I faced was really slow finetuning and low BLEU scores. Increasing the batch size from 2 to 4 helped regarding both of these problems, since it allowed for a smoother gradient descent and more utilization of resources. I was not able to increase batch size more than 4 though since Collab was running out of memory. Another reason for getting low scores was I was not evaluating my translations properly. Upon debugging the code I found the input to the BLEU score calculator function had extra information other than just the pure French translated sentences. After fixing this issue my score jumped from around 3 to 10, which was a big improvement.

3. **What adjustments to your design and implementation were necessary during the implementation phase? What would you change or do differently if you had to do the portfolio task a second time? What would be potential areas for future improvement?**
   One thing I faced when generating synthetic data was, I initially thought we had to load the larger model and then perform inference. I was not able to load the Qwen32B model in Collab. Then someone mentioned to use API, but that was also an issue since it only worked for a limited amount of times. In order to solve this problem I had to create batch's of 5 sentences per API call. Anymore than this the model started generating really bad sentences or sometimes didn't even generate anything. Something I would do differently again would be not running on so many epochs. I ran all my experiments on 15 epochs, but after seeing all the results, all of them plateau out after max 10 epochs. This would've helped me finetune faster and save more time.
4. **Include a brief section on ethical considerations when using these models on language translation tasks.**
   When using LLMs for language translation, ethical concerns such as bias, misinformation, and privacy risks must be considered. Models may inherit biases from training data, leading to unfair or inaccurate translations, especially for underrepresented languages. Additionally, privacy concerns arise when translating sensitive information, making data security crucial.
5. **From the lecture/course including guest lectures, what topic excited you the most? Why? What would you like to learn more about and why?**
   I really liked the second assignment which introduced us to the concept of RAG. I also really liked how they taught us up to date things.
6. **How did you find working with DIFY platform during the course work? Would you recommend using DIFY in learning Generative AI technologies and why? What is the best start for learning Generative AI either by Python code or No-code platforms and why?**
   DIFY was fun to work with, but I thought it had many limitations as of now and wasn't really flexible as much as I would like it. I wouldn't really recommend it though. For me learning Gen AI with Python will still trump no code platforms. I think Python allows you to be so much more flexible while also being easy to start off with. However we are heading towards no code platforms now so maybe soon I'll be forced to change my mind.
7. **How did you find the assignments and exercises in the course and how they help you in portfolio exam?**
   As I mentioned above I really liked the second assignment about RAG, since it was a topic I wanted to learn about for quite some time now but wasn't getting the chance to. It introduced it in a really nice step by step way. The first assignment would've helped me in the portfolio exam also, had I chosen the other task.